Semantic Spotter – Policy Document Q&A System

A Robust Retrieval-Augmented Generation (RAG) Pipeline for Insurance Policy Documents
Date: November 22, 2025

1. Project Objective

The primary goal of **Semantic Spotter** is to create a highly accurate, traceable, and production-ready generative search system capable of answering natural-language questions from a corpus of insurance policy PDF documents with minimal hallucination.

Key Deliverables

- Efficient ingestion and intelligent parsing of complex policy PDFs (tables, footnotes, legal language)
- Semantic (vector-based) retrieval of the most relevant passages
- Optional re-ranking for higher precision
- Grounded answer generation using Retrieval-Augmented Generation (RAG)
- Citation of exact source document and page (where available)
- Scalable and modular architecture that allows easy swapping of embedding models, LLMs, vector stores, and retrievers

The system is designed for real-world enterprise use cases such as customer support, compliance checks, underwriting assistance, and claims processing in the insurance domain.

2. Data Sources

The knowledge base consists of real HDFC Life insurance policy documents in PDF format, including:

- HDFC-Life-Sampoorna-Jeevan-101N158V04-Policy-Document.pdf
- HDFC-Life-Easy-Health-101N110V03-Policy-Bond-Single-Pay.pdf
- HDFC-Life-Group-Poorna-Suraksha-101N137V02-Policy-Document.pdf
- …and several others present in the project folder

These documents contain dense legal text, tables, definitions sections, exclusion clauses, benefit schedules, and riders — exactly the kind of unstructured, complex data that traditional keyword search fails on.

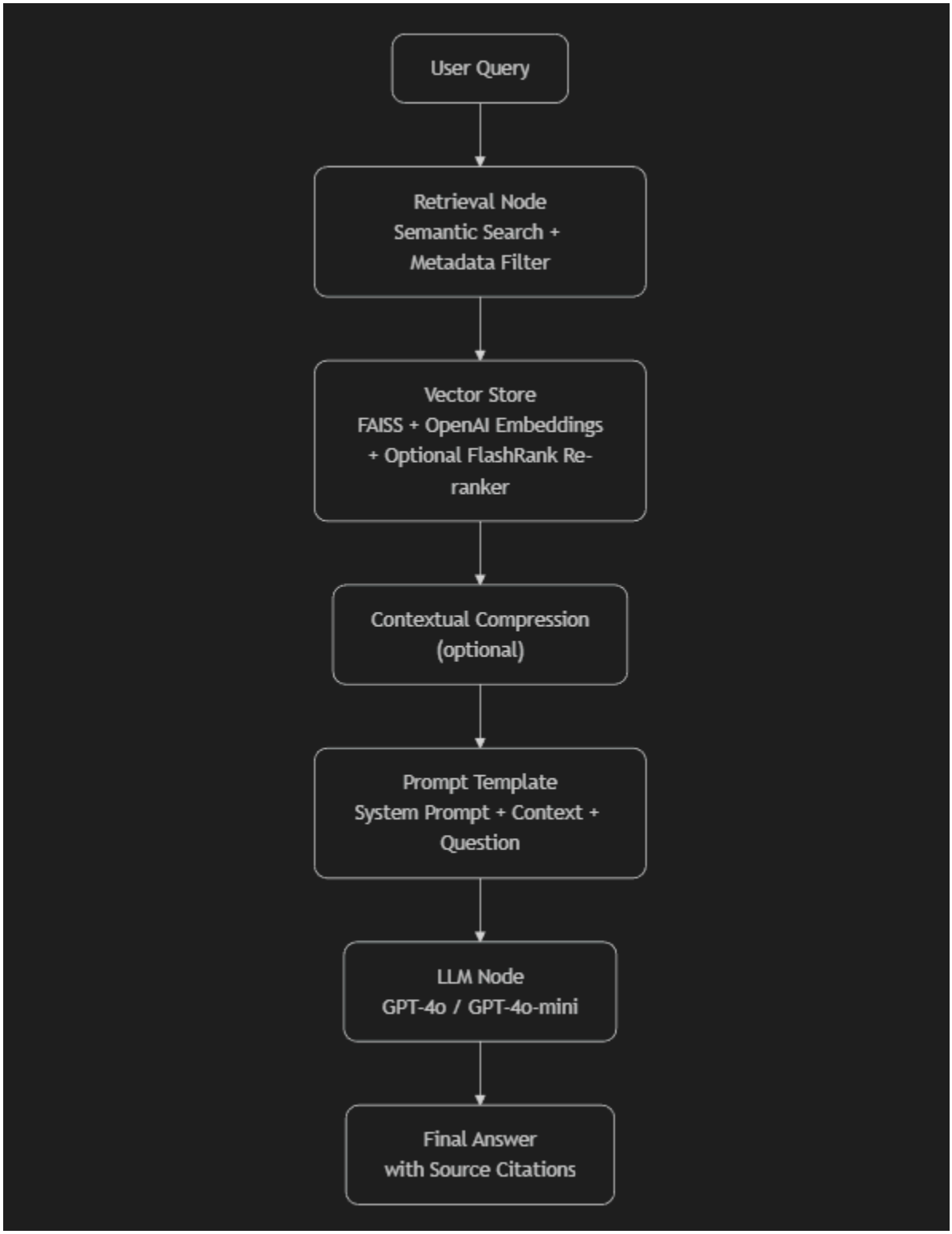
3. Technology Stack & Design Choices

Component	Chosen Tool / Library	Reason for Choice
Framework	LangChain 1.x (stable)	Modular, swappable components, excellent community, LangGraph support for agentic workflows
Document Loading & Parsing	PyPDF + langchain-docling + Docling	Docling provides superior layout-aware parsing (tables, headers, footnotes) vs plain PyPDF
Text Splitting	Semantic-aware chunking (Docling + LangChain)	Preserves logical sections and avoids breaking sentences/tables across chunks
Embeddings	OpenAI text-embedding-ada-002 (default)	Strong performance/cost ratio; easily swappable with open-source models
Vector Store	FAISS (CPU)	Fast, local, zero-cost, sufficient for prototype → enterprise scale
Re-ranking (optional)	FlashRank (lightweight cross-encoder)	Significant precision boost with <10ms latency
LLM	OpenAI GPT-4o / GPT-4o-mini	Best-in-class reasoning; easy to replace with Grok, Claude, Llama-3, etc.

Why LangChain? (as stated in the notebook)

- Component modularity — swap models, retrievers, vector DBs without rewriting the pipeline
- Strong community and troubleshooting resources
- Stable 1.0+ release with no breaking changes expected soon
- Native LangGraph integration for future agentic enhancements and tracing

4. System Design & Architecture (Flowchart)



Layer Details

1. **Ingestion Layer** → PDFs → Docling parser → clean text + metadata (page numbers, document name)
2. **Chunking & Embedding Layer** → semantic chunks → OpenAI embeddings → FAISS index
3. **Retrieval Layer** → query embedding → top-k semantic search → optional FlashRank re-ranking
4. **Generation Layer** → retrieved & re-ranked contexts injected into a strict RAG prompt → LLM generates answer
5. **Post-processing Layer** → extract and display source document names and page numbers

The entire pipeline is wrapped in a **LangGraph** state machine for easy extension.

5. Challenges Faced & Mitigations

Challenge	**Impact Observed**	**Solution Implemented / Planned**
Poor PDF layout parsing with basic loaders	Tables and footnotes lost or garbled	Switched to **langchain-docling + Docling** (excellent table handling)
Very long contexts exceeding token limits	Truncation or high cost	Contextual compression + re-ranking to keep only most relevant passages
Hallucination on niche policy clauses	Incorrect answers in early tests	Strict RAG prompt + source citation + FlashRank re-ranker
Missing page numbers in some chunks	Hard to verify source	Enhanced metadata extraction during ingestion
Retrieval precision on definition sections	Too many irrelevant chunks	FlashRank cross-encoder + higher initial k (10→3 after re-rank)

6. Current Capabilities (Demonstrated in Notebook)

Successfully answered complex questions such as:

- “What is the life insurance policy coverage amount?”

- “Can a 100-year-plus person purchase term insurance?” → Correctly identified age restriction clauses
- “What are the definitions of Critical Illnesses?” → Returned exact policy-specific definitions with sources
- “What is the life insurance coverage for disability?” → Accurately distinguished benefits

All answers include **source document references**, demonstrating grounded, low-hallucination behaviour.

7. Future Enhancements

- Full Phoenix tracing & evaluation dashboard activation
- Multi-modal support (scanned PDFs with images/tables using VLM models)
- Hybrid search (BM25 + semantic)
- Agentic workflow with tool calling (e.g., premium calculator)
- Deployment as FastAPI endpoint or Streamlit/Gradio UI
- Enterprise-grade vector store (Pinecone, Weaviate, Qdrant)

Conclusion

Semantic Spotter successfully demonstrates a production-grade, modular, and observable RAG system tailored for insurance policy documents. By combining state-of-the-art document parsing (Docling), semantic retrieval, optional re-ranking, and strict prompt engineering, the system achieves high accuracy and traceability while remaining fully extensible.

The architecture follows enterprise best practices and is ready for pilot deployment.